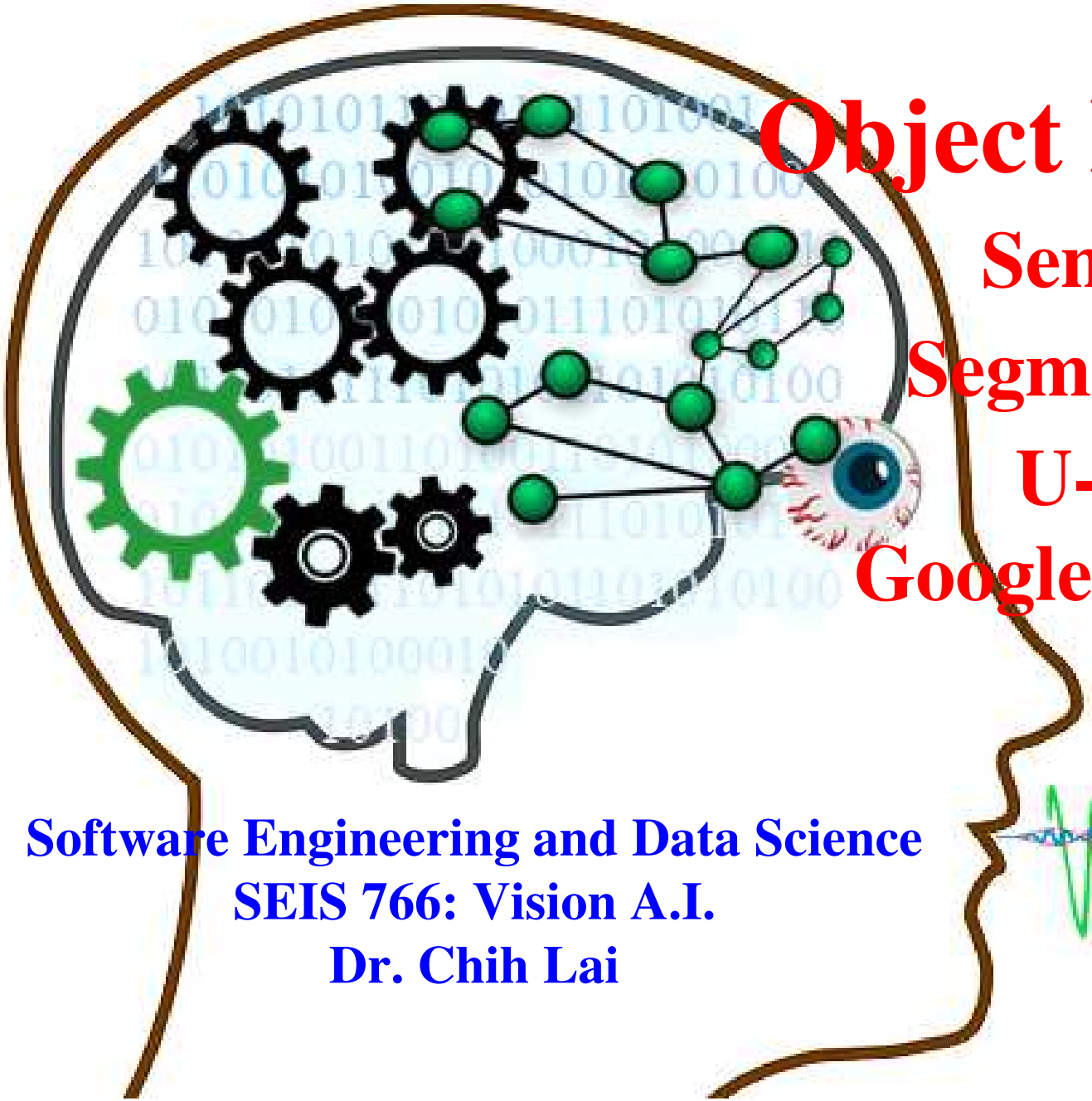


Object Detection

Semantic Segmentation

U-Net / Google DeepLab

Software Engineering and Data Science
SEIS 766: Vision A.I.
Dr. Chih Lai

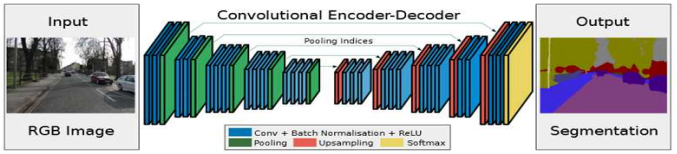
Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
  - **SegNet, U-Net** Family, **Google's DeepLab V3+, Meta's SAM**
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map

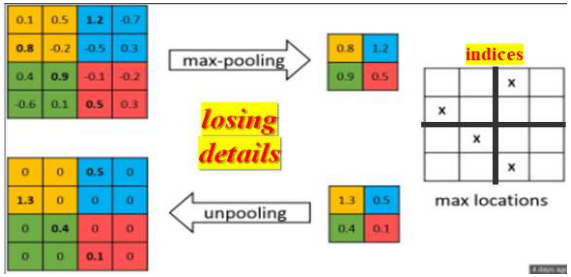
Transfer Learning Frameworks for Semantic Segmentation

- SegNet, 2015
- U-Net 2015, U-Net++ 2019, U-Net3+ 2020
- Google, DeepLabV3+, 2018
- Meta, Segment Anything Model (SAM) (2023)
- **They are for different purposes, not just a linear upgrade over years**
- EfficientUNet (2020), Segmenter (2021), SegFormer (2021)

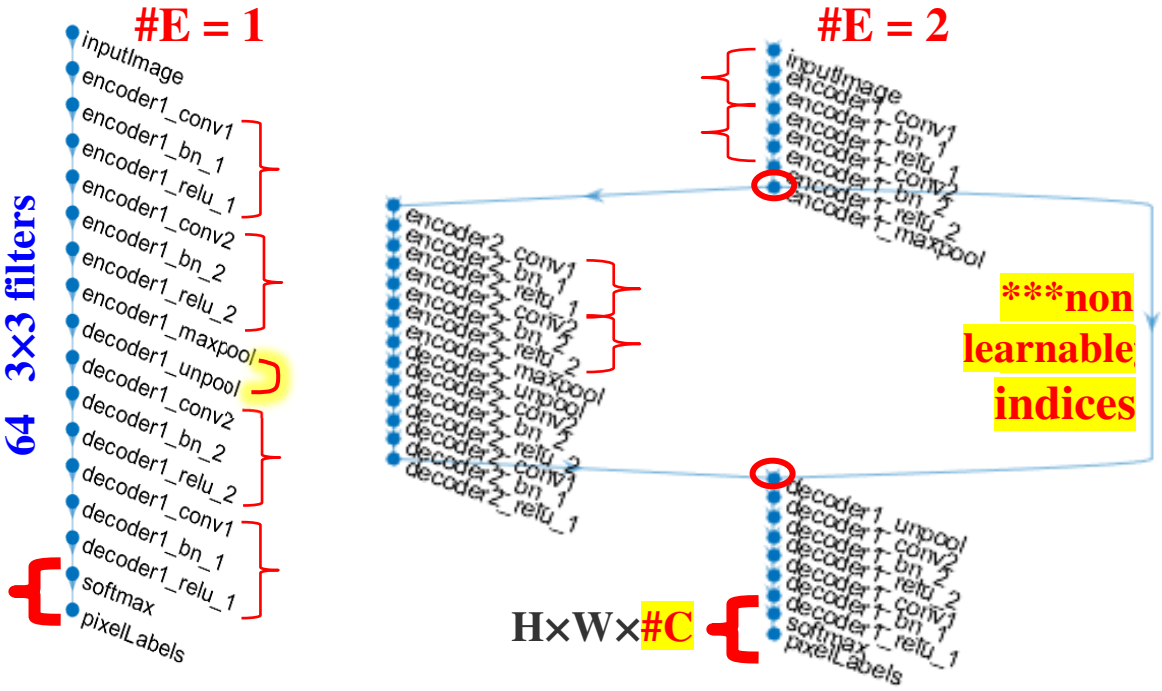
SegNet in Matlab



- **segnetLayers**(inputImageSize, numClasses, **#encoders**)
  - Each encoder layer convolute the input **twice**
  - **VGG16-like** w/ **DeConv** + **max-un/pooling** w/ **indices**
    - because pooling may lose fine details



#encoders (or decoders) layers =



https://www.mathworks.com/help/vision/ref/segnetlayers.html#d120e117005

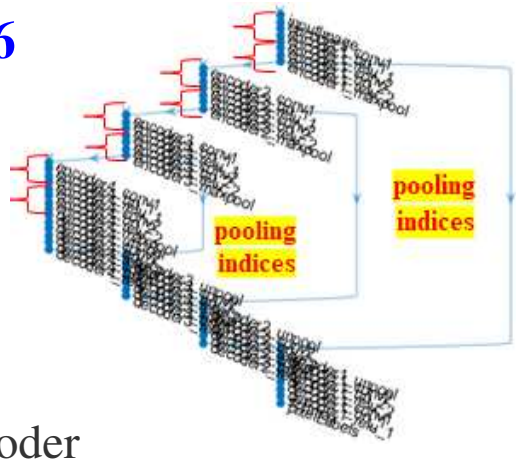
```
lgraph = segnetLayers([224 224 3], numClasses = 2, 'vgg16');
analyzeNetwork(lgraph)
```

Keras has no built-in **SegNet**

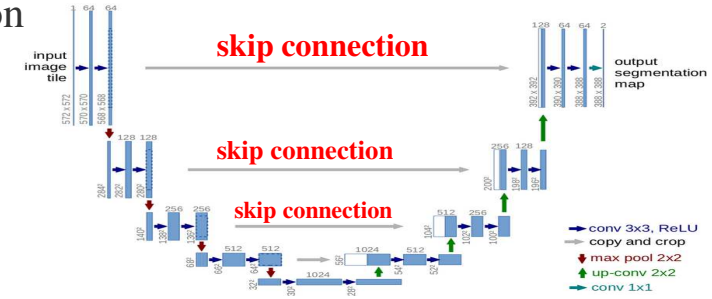
https://www.kaggle.com/code/harlequeen/semantic-segmentation-of-plants-with-segnet

Better Pre-Trained Semantic Segmentation Networks on **Keras**

- SegNet has **pre-trained for segmentation** based on **VGG-16**
  - SegNet uses "***pooling***" with **pooling *indices***
  - https://www.mathworks.com/help/vision/examples/semantic-segmentation-using-deep-learning.html
  - https://www.mathworks.com/help/vision/ug/semantic-segmentation-basics.html



- **U-Net *family*** 2015
  - U-Net uses "**skip-connection**" to pass **F.M.** between decoder / encoder
  - Still "***pooling***", use in many medical apps & stable diffusion
  - **Keras**, Image segmentation with a **U-Net-like** architecture
    https://keras.io/examples/vision/oxford_pets_image_segmentation/
    https://www.mathworks.com/help/vision/ref/unet.html



https://towardsdatascience.com/unet-line-by-line-explanation-9b191c76baf5

- **Google DeepLab V3+** 2018
  - DeepLab uses **less aggressive** "***pooling***" with ***atrous*** convolution
  - Use ***atrous*** **convolution** to preserve better image details **(FOV field of view) w/o** heavy computing

https://**keras**.io/examples/vision/deeplabv3_plus/ **https://www.mathworks.com/help/vision/ref/deeplabv3plus.html**

- **SAM???** **2023 / 2025**

Comparison, All to Preserve Image Detail for Precise Object Locations

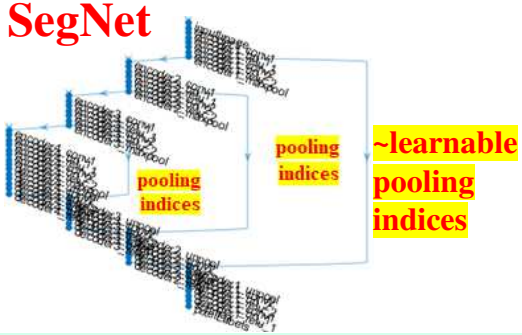
- **They are for different purposes, not just a linear upgrade over years**

Feature	DeepLabV3+ (2018)	U-Net3+ (2015 / 2019 / 2020)	SAM / Segment Anything (2023)
Architecture Type	CNN-based, NO decoder, with **dilated convolutions** / **Atrous** Convolutions	CNN **encoder–decoder** with **full-scale skip connections** (dense connections across all scales)	Transformer-based **ViT** with multiple **encoder** + lightweight mask **decoder**
Generalization	Moderate → needs retraining/fine-tuning for new domains	Moderate → needs retraining/fine-tuning for new domains	Very high → **zero-shot** or prompt-based segmentation **(but = regions)**
Prompting	No user prompt required (other than labels)	No user prompt required (other than labels)	Supports user prompts (points, boxes) for **interactive *region* segmentation**
Parameter Size	~40–60M (depending on backbone)	~25–40M	Very large: **~600M** (ViT-H encoder) Trained on 11M images, 1 billion labels, high resolution images
Strengths	Strong semantic segmentation; robust to scale variation (via ASPP); good for natural images **Faster** for real-time surgery w/ upgrade	Excellent multi-scale fusion; strong performance on biomedical & fine-structure segmentation	Foundation model; **highly generalizable** for **regions**; interactive segmentation; strong zero-shot ability
Weaknesses	Rough boundary precision; No instance separation	**Heavy computing**; slower due to **full-scale skip,** No instance separation	**Very large model**, **memory intensive**; **domain gap** on medical images without adaptation, No instance separation

UNet, *S*kip *C*onnections (SCs), Encoder (*E*) / Decoder (*D*)

- Encoder's (*E*) uses down-sampling to capture/summarize important image content
- But to allocate objects in *D*, *D* needs to recover details lost in *E*'s down-sampling
- **SC**s for: **1. vanishing gradient** (*VG*) & **2. preserve detailed info.** from earlier layers
  1. **VG** = repeatedly multiply small derivatives of activation functions in earlier
  1. **SC**s provide **shortcuts for gradient to flow** from later layers back to earlier layers
  2. **SC**s copy ***FM***s from ***E***'s layers to ***D*** to be ***stacked*** (concatenate) w/ ***FM***s from the ***D***'s last layer

SegNet

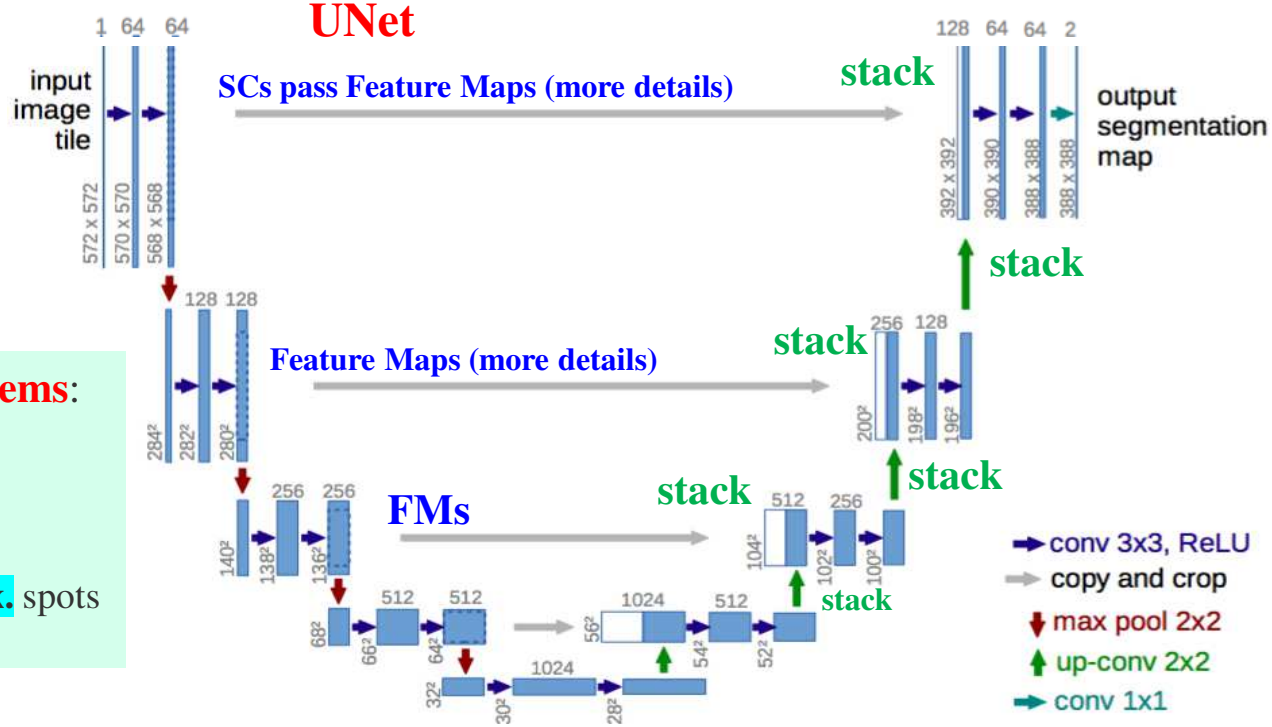


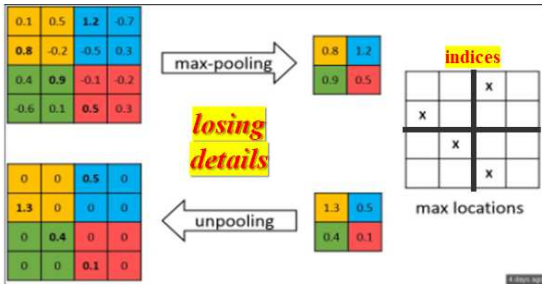
Pooling index / connection, problems:

1. No gradient backward
2. Carry NO detail info.

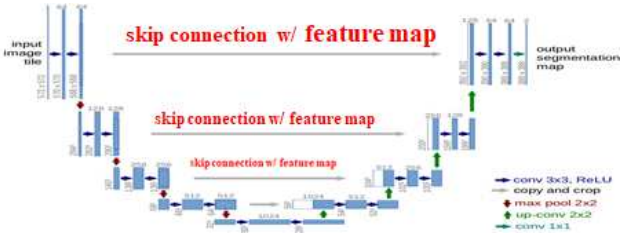
   Only restores reduced info to **approx.** spots

UNet





SegNet vs. U-Net



Feature	SegNet	U-Net
Architecture	Encoder-Decoder with max-pooling indices for unpooling	U-shaped Encoder-Decoder with skip connections
Upsampling Method	Unpooling using max-pooling indices	Transposed convolutions with skip connections
Memory Efficiency	More memory-efficient by storing only max-pooling indices	Requires more memory due to full feature map connections
Precision	Good, but depends on the accuracy of the pooling indices	High precision due to skip connections and direct feature transfer
Primary Use Case	Autonomous driving, scene understanding	Medical imaging, general-purpose segmentation
Strengths	Lower memory usage, efficient unpooling	High accuracy for fine boundary segmentation
Weaknesses	*May lose fine details*	*Higher memory requirements* due to feature map transfers

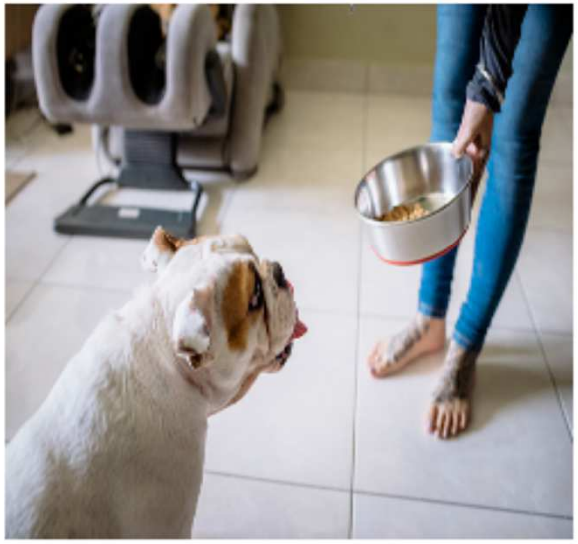
pass ***F.M.***s from ***E*** to ***D***, help recover lost details

also used in DeepLab V3

Sample UNet Segmentation, Implementation & Results

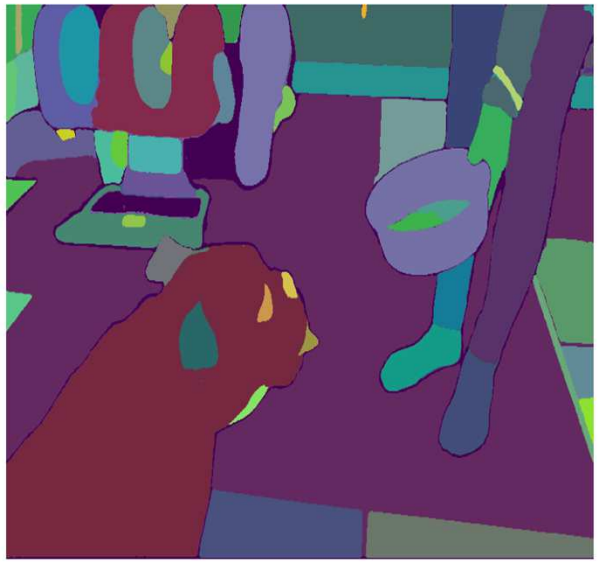
VAI_Slide_UNet_SegDog.ipynb

Raw Image (Input)



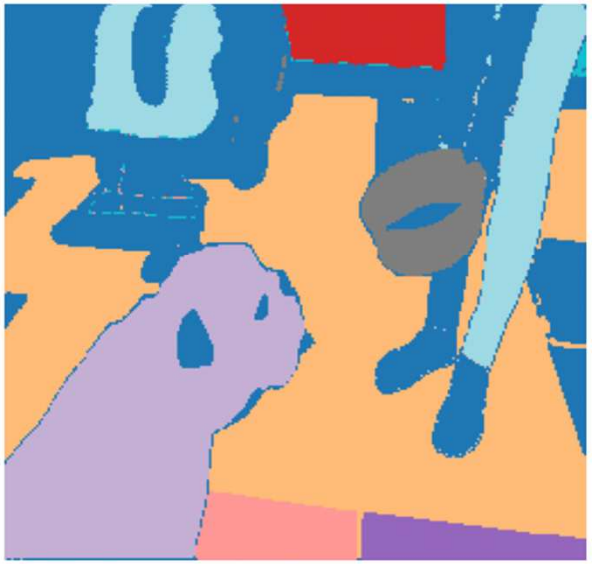
Dog.png

Ground Truth (Colored)



Dog_SAM_Labels.png

Prediction (Colored)
IoU: 0.08



U-Net, Pass Feature Maps on Skip Connections

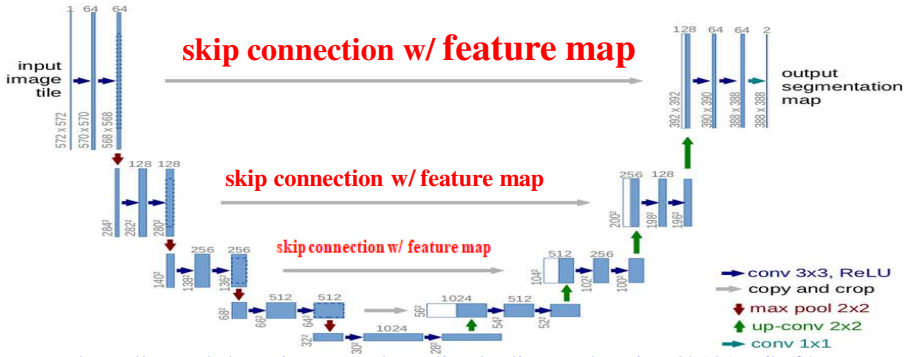
- UNet **skip-conn** carry more details than indices in SegNet
  - **skip-conn** pass spatial info in **FM**s from ***E***ncoder to ***D***ecoder
  - **skip-conn** needs higher memory, so SegNet is faster
  - # conv layers = $2^{\text{Enc\_depth}} + 2$

```
%% https://www.mathworks.com/help/vision/ref/unetlayers.html#namevaluepairarguments
imageSize = [32 32];                    numClasses = 2;
lgraph = unet(imageSize, numClasses, 'EncoderDepth', 1, ...
                'FilterSize', 5, 'NumFirstEncoderFilters', 16)
```

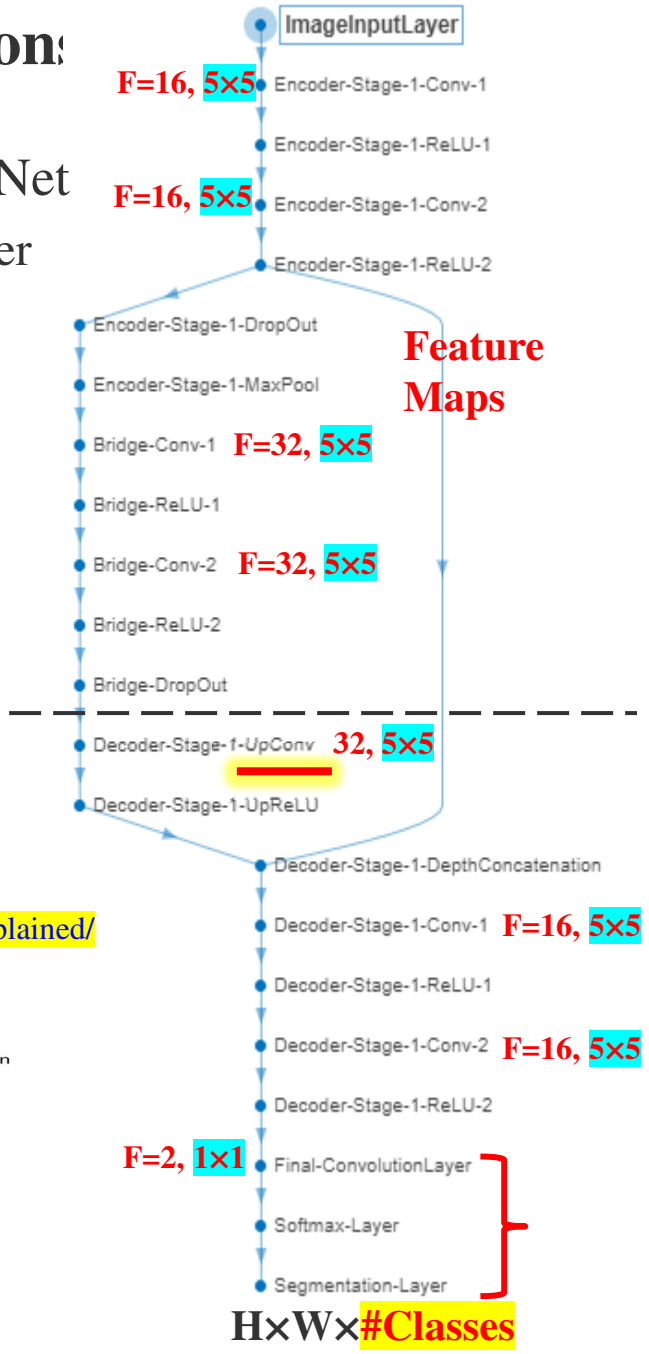
- U-Net-like architecture

VAI_Slide_UNet_SegDog.ipynb

  - https://keras.io/examples/vision/oxford_pets_image_segmentation/
  - **Easy to read py**: https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/



https://towardsdatascience.com/unet-line-by-line-explanation-9b191c76baf5



UNet Implementation… *especially*, Skip Connection

```
def conv_block(inputs, num_filters):
    # Standard Double Convolution Block
    x = layers.Conv2D(num_filters, 3, padding="same")(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    x = layers.Conv2D(num_filters, 3, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    return x
```

1 2 3 4

VAI_Slide_UNet_SegDog.ipynb

```
def encoder_block(in, #filters):
    ppf = conv_block(in, #filters)  # pre-pool
    p = layers.MaxPool2D((2, 2))(ppf)
    return ppf, p   # ppf = PP-FM, p = pooling
```

```
def decoder_block(inputs, skip_features, num_filters):
    # Upsample
    Z = layers.Conv2DTranspose(num_filters, (2, 2), strides=2, padding="same")(inputs)
    # Copy and Crop (Skip Connection)
    Z = layers.Concatenate()( [Z,  skip_features] )
    # Conv Block
    Z = conv_block(Z, num_filters)
    return Z
```

Skip Conn.. send pre-pooled FM

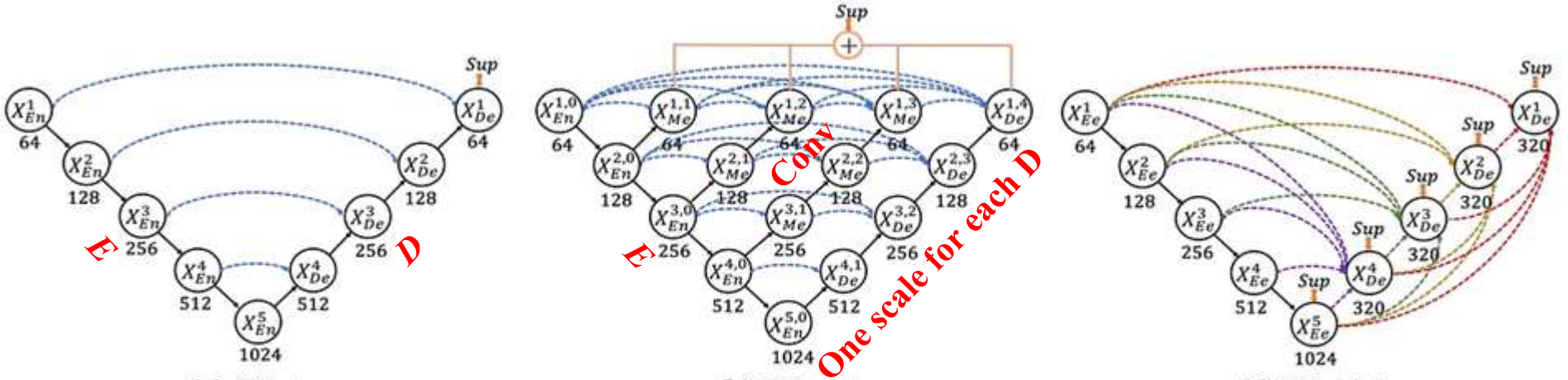
```
def build_unet(input_shape, num_classes):
    inputs = layers.Input(input_shape)
    # --- Encoder (Downsampling) ---
    s1, p1 = encoder_block(inputs, 64)    # Out 128x128
    s2, p2 = encoder_block(p1, 128)       # Out 64x64
    s3, p3 = encoder_block(p2, 256)       # Out 32x32
    s4, p4 = encoder_block(p3, 512)       # Out 16x16

    # --- Bridge (Bottleneck) ---
    b1 = conv_block(p4, 1024)             # Out 16x16x1024

    . . . . . .

    # --- Decoder (Upsampling) ---
    d1 = decoder_block(b1, s4, 512)       # In: 16x16 + 16x16 -> Out: 32x32
    d2 = decoder_block(d1, s3, 256)       # In: 32x32 + 32x32 -> Out: 64x64
    d3 = decoder_block(d2, s2, 128)       # In: 64x64 + 64x64 -> Out: 128x128
    d4 = decoder_block(d3, s1, 64)        # In: 128x128 + 128x128 -> Out: 256x256

    . . . . . .

    outputs = layers.Conv2D(num_classes, 1, padding="same", activation="softmax")(d4)
    model = models.Model(inputs, outputs)
    return model
```

1 2 3 4

1×1 to compress channels

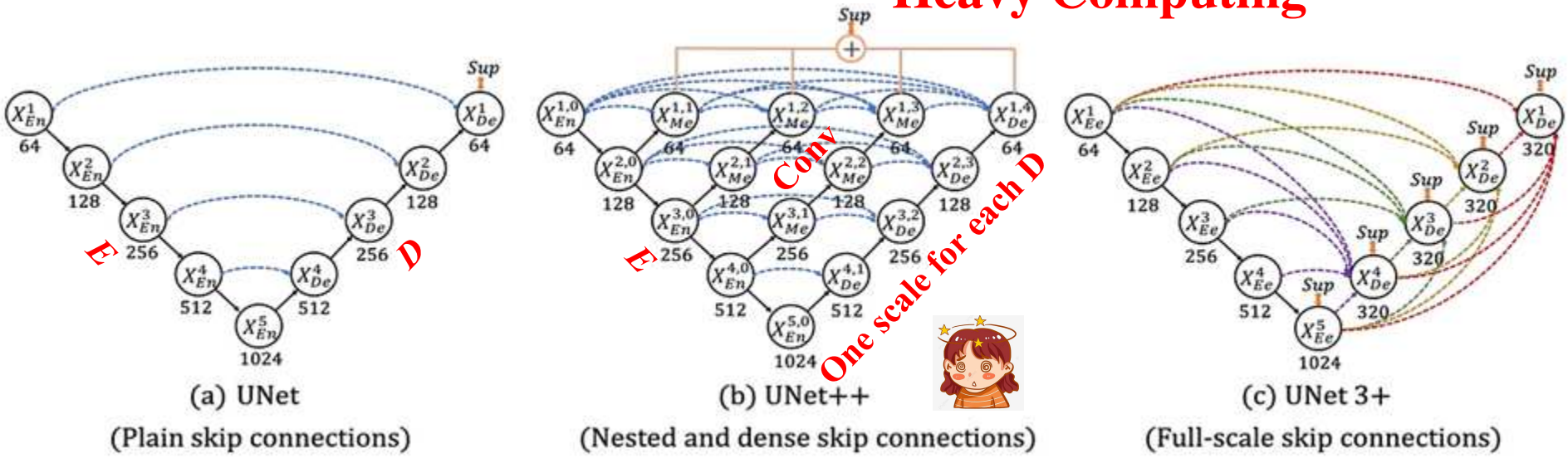
UNet, UNet++, UNet3+

- **UNet:** ***E/D*** architecture w/ skip connections to capture both global & local details
  - Pass ***FM***s from ***E*** to ***D*** **at same depth**
- **UNet++:** Use dense & nested ***Conv***s to refine ***FM***s before giving to ***D*** **at same depth**
  - Progressive refinement of ***FM***s
- **UNet3+:** Resize ***FM***s from all scales (layers) of ***E*** & fuse w/ ***D***'s ***FM***s at each ***D***'s layer
  - Not just ***FM***s from the same depth ***E*** layer, instead ***all*-scale *SC*s**, need resizing
  - Multi-scale ***SC***s collects ***FM***s **from all *E*'s layers**, stack w/ ***FM***s from all other ***D***'s layers
  - Aggregate all scales at every decoder stage to maximize multi-scale info



UNet, UNet++, UNet3+, Summary Table

Heavy Computing



(a) UNet (Plain skip connections)

(b) UNet++ (Nested and dense skip connections)

(c) UNet 3+ (Full-scale skip connections)

Model		Skip connection type	Connection pattern	Main idea
U-Net	2015	Long skip	Encoder ↔ Decoder (same level)	Direct feature reuse, sharp details
U-Net++	2019	Dense skip (nested)	Encoder ↔ Decoder with intermediate conv blocks	Narrow semantic gap, better fusion
U-Net3+	2020	Full-scale skip	Multi-scale encoder + decoder → Decoder node	Capture both local and global context

Not Learned

Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
  - **SegNet, U-Net** Family, **Google's DeepLab V3+,** √ **Meta's SAM**
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map

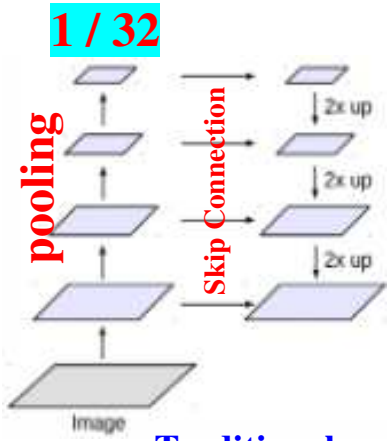
Google DeepLab V3+ / Semantic Segmentation, WHY???

Folder: \FishEmbryo_Semseg_Programs\

VAI_Slide_deeplabv3_plus_FishEmbryo.ipynb

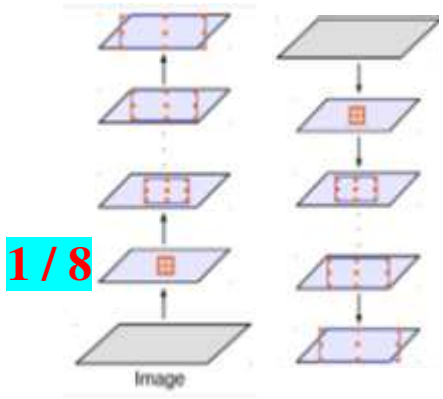
VAI_Slide_FishEmboyo_Deeplab_V3_Train.m

- Still Encoder / Decoder architecture
- We know ***pooling*** causes ↓resolution **&** info loss
  - OK for classifying images, **bad** for pixel segmentation
- **Google solution?? Atrous** ('**trous**' in French = "holes") **Convolution**
- What is this? **with less aggressive "pooling"**

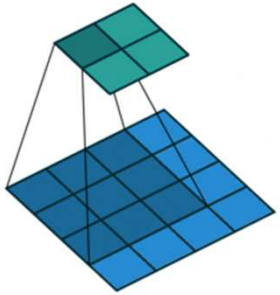


Traditional
Encoder-Decoder
"pooling" shrink F.M.
➔ info loss

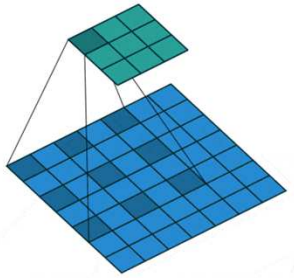
Astrous Convs



Astrous Convs
maintains F.M.
w/ larger FOV
(Field of View)



Traditional
Conv

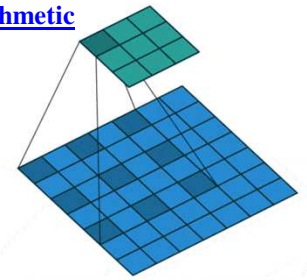


Astrous
Conv

https://github.com/vdumoulin/conv_arithmetic

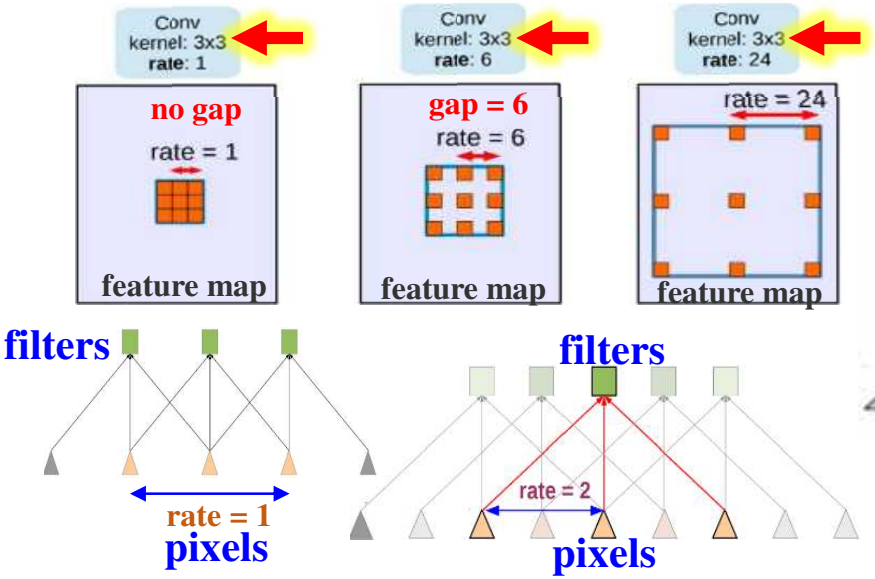
Atrous Convolution

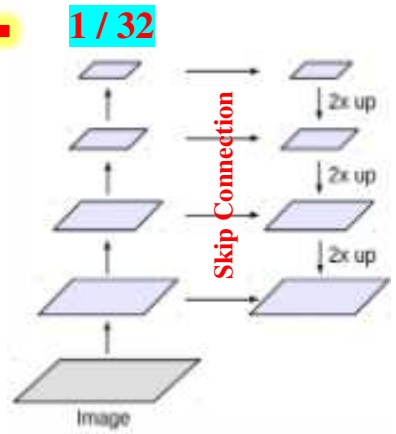
https://github.com/vdumoulin/conv_arithmetic



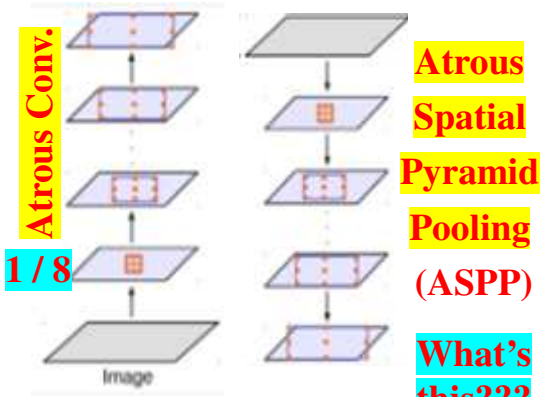
- Known as ***dilated convolution*** = filters read pixels **with gaps**
  - To get larger view, regular conv. need too many connections➔ ↑θs & slow
  - Enlarge FOV conv. filters **w/o pooling** or adding too many θs for computations
  - Cheaper to work on high resolution maps & extract features from **different FOVs**

https://learnopencv.com/deeplabv3-ultimate-guide/





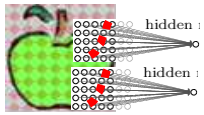
Encoder-Decoder
Convs shrink F.M.
& info loss



Astrous Convs
maintain F.M.
w/ larger FOV

Atrous Spatial Pyramid Pooling (ASPP)

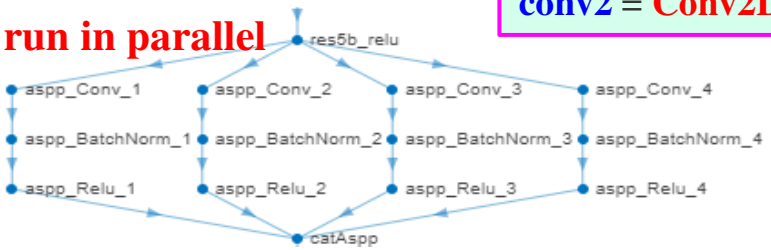
What's this???



convolution2dLayer(filterSize, numFilters, DilationFactor=2)

```
# ALL dilated convs. run in parallel
conv1 = Conv2D(filters=32, kernel_size=3, dilation_rate=6)
conv2 = Conv2D(filters=64, kernel_size=3, dilation_rate=24)
```

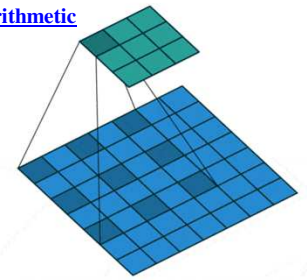
run in parallel



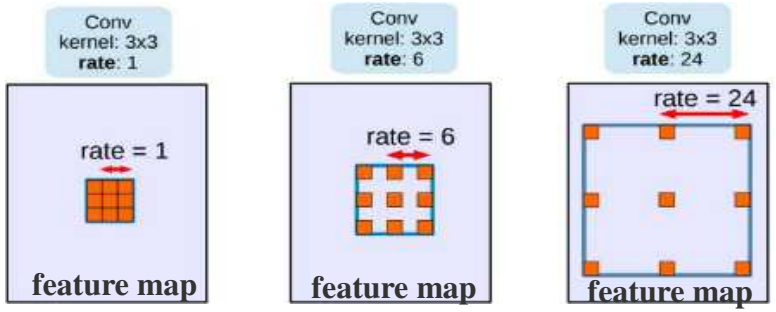
Then, just insert dilated conv. in later layers of a pretrained CNN

Atrous Spatial Pyramid Pooling (ASPP)

https://github.com/vdumoulin/conv_arithmetic

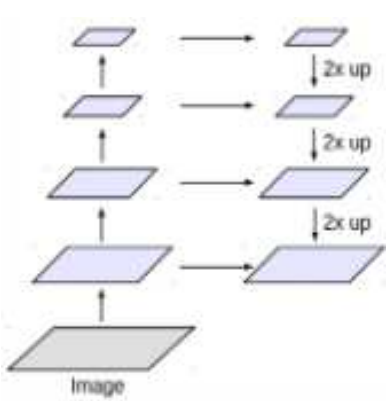


- Known as ***dilated convolution*** = filters read pixels **with gaps**
  - Enlarge FOV conv. filters **w/o pooling** or adding too many θs for computations
  - Cheaper to work on high resolution maps & extract features from **different FOVs**
- Need **Atrous Spatial Pyramid Pooling (ASPP)** to assemble different size feature maps
  - "***pooling***" is a bit **misleading**, **ASPP =** just places F.M.s at correct locations

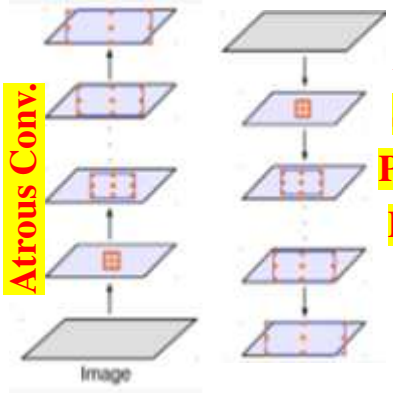


https://learnopencv.com/deeplabv3-ultimate-guide/

```
# ALL dilated convs. run in parallel
conv1=Conv2D(filters=32, kernel_size=3, dilation_rate=6)
conv2=Conv2D(filters=64, kernel_size=3, dilation_rate=24)
```



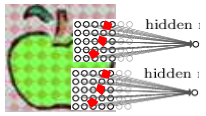
Encoder-Decoder Convs shrink F.M. & info loss

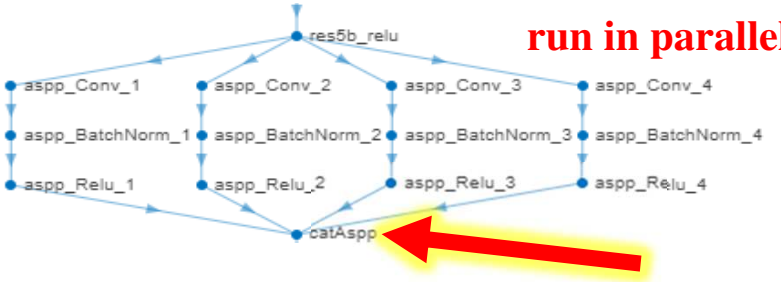


Astrous Convs maintain F.M. w/ larger FOV

Atrous Spatial Pyramid Pooling (ASPP)

What's this???

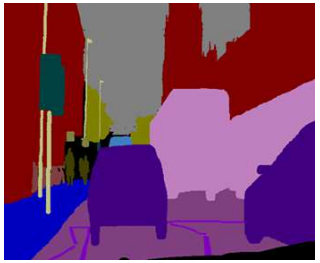




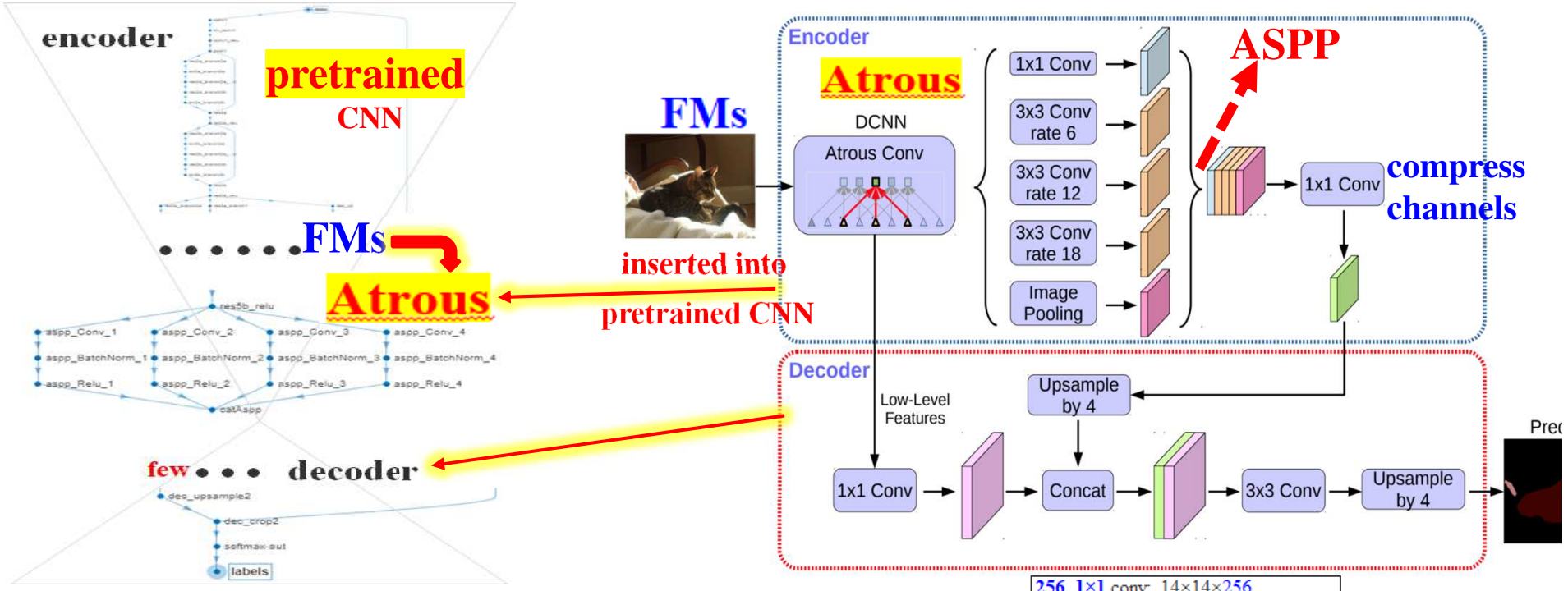
run in parallel

Just insert dilated conv. in later layers of a pretrained CNN

Google DeepLab V3+ for Semantic Segmentation



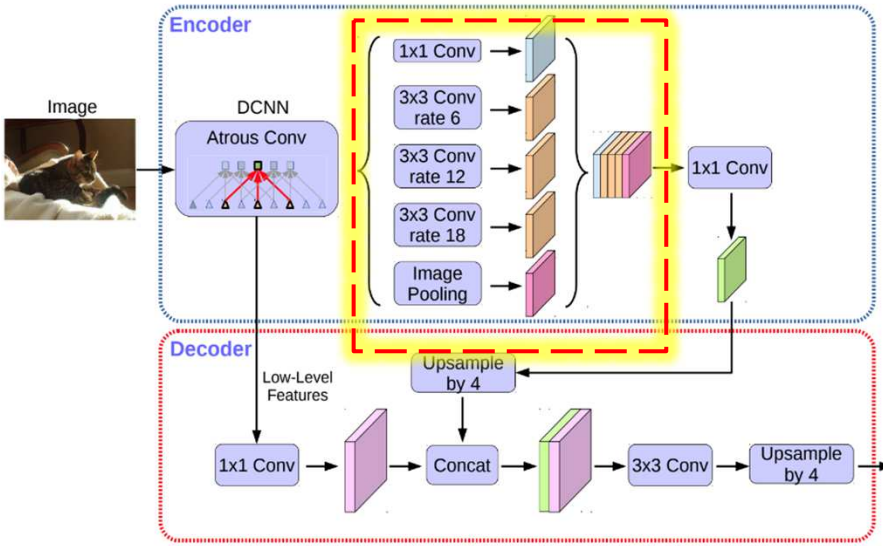
- Encoder / Decoder architecture w/ **Atrous** Conv.
- Modular design easy to be adapted in **pre-trained CNN** like ResNet
  - Atrous process **feature maps** (**FMs**) from **last** few layers of a pretrained CNN
  - Different pretrained CNNs may adapt Atrous conv slightly differently

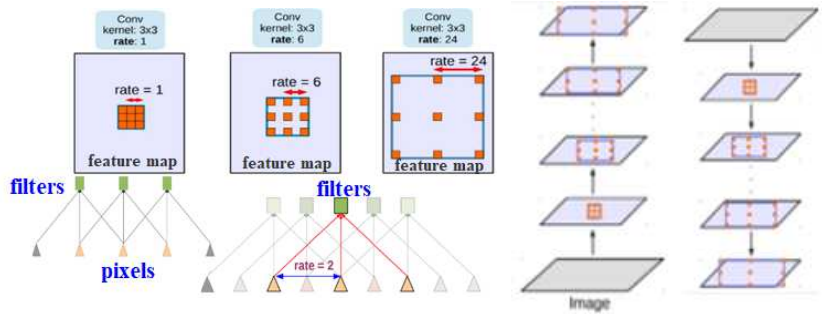



```
# ALL dilated convs. run in parallel
conv1 = Conv2D(filters=32, kernel_size=3, dilation_rate=6)
conv2 = Conv2D(filters=64, kernel_size=3, dilation_rate=18)
```

256, 1×1 conv: 14×14×256
256, 3×3 conv: 14×14×256 (learning)
Upsample by 16: 224×224×256
3, 1×1 conv: 224×224×3 classes
Softmax: 224×224×3

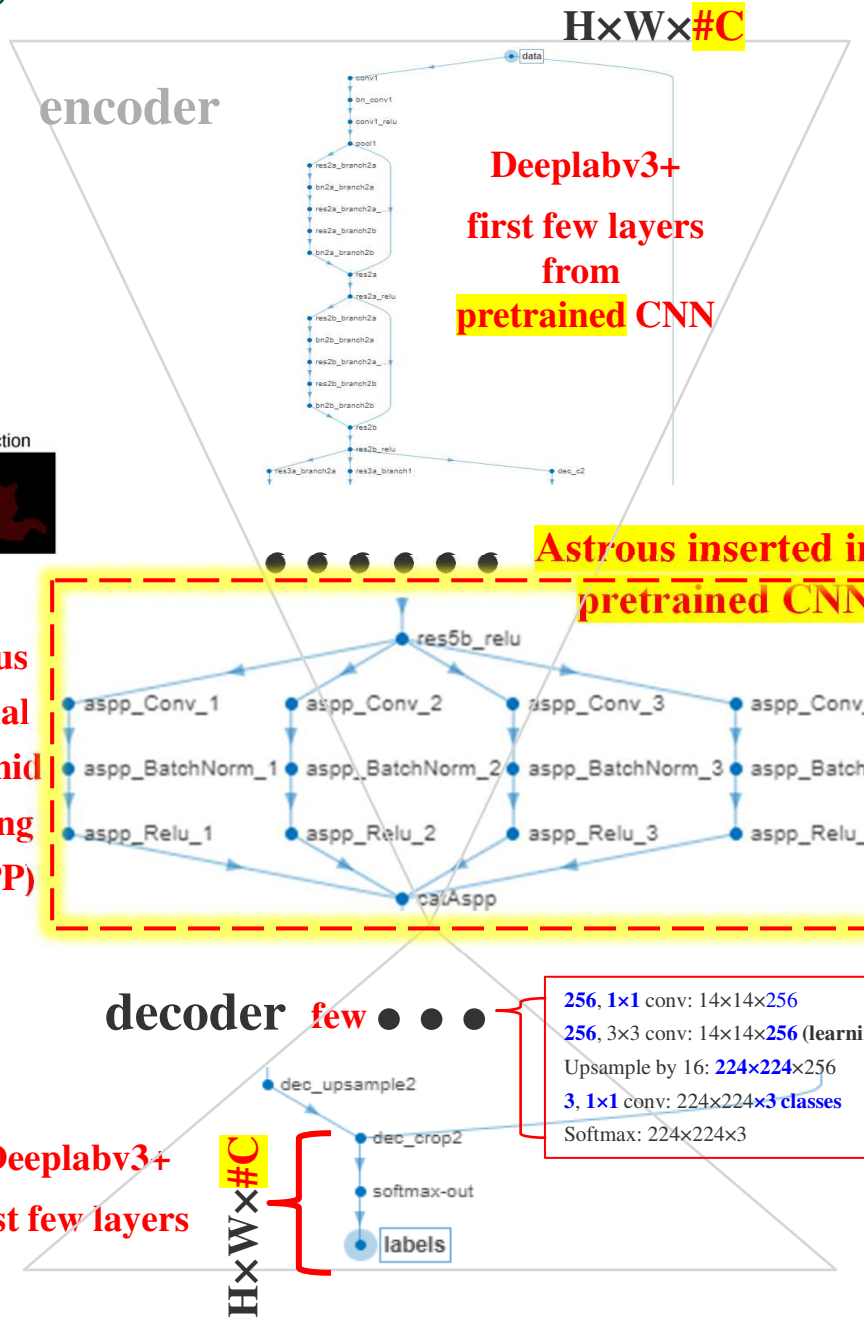
Overall Architecture of DeepLab V3+





Atrous Spatial Pyramid Pooling (ASPP)

H×W×#C



Folder: \FishEmbryo_Semseg_Programs

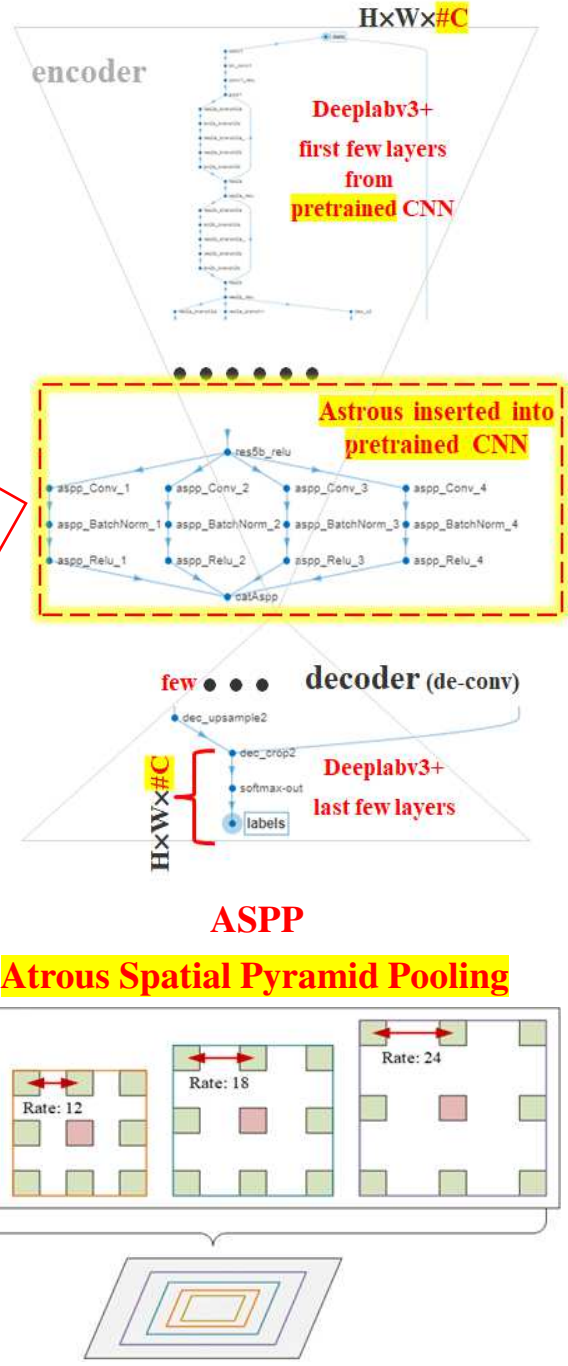
VAI_Slide_deeplabv3_plus_FishEmbryo.ipynb

VAI_Slide_FishEmboyo_Deeplab_V3_Train.m

VAI_Slide_deeplabv3_plus_FishEmbryo.ipynb

```
def DilatedSpatialPyramidPooling(dspp_input):
    dims = dspp_input.shape
    x = layers.AveragePooling2D(pool_size=(dims[-3], dims[-2]))(dspp_input)
    x = convolution_block(x, kernel_size=1, use_bias=True)
    out_pool = layers.UpSampling2D(
        size=(dims[-3] // x.shape[1], dims[-2] // x.shape[2]),
        interpolation="bilinear",
    )(x)
    out_1 = convolution_block(dspp_input, kernel_size=1, dilation_rate=1)
    out_6 = convolution_block(dspp_input, kernel_size=3, dilation_rate=6)
    out_12 = convolution_block(dspp_input, kernel_size=3, dilation_rate=12)
    out_18 = convolution_block(dspp_input, kernel_size=3, dilation_rate=18)

    x = layers.Concatenate(axis=-1)([out_pool, out_1, out_6, out_12, out_18])
    output = convolution_block(x, kernel_size=1)      % 1x1 conv
    return output
```

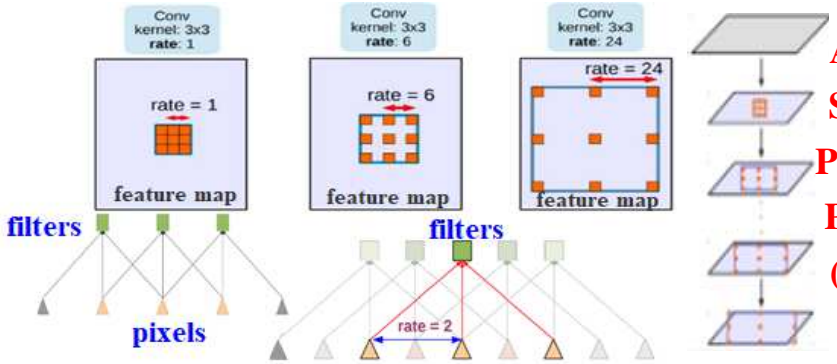
```
def DeeplabV3Plus(image_size, num_classes):
    model_input = keras.Input(shape=(image_size[0], image_size[1], 3))
    resnet50 = keras.applications.ResNet50(
        weights="imagenet", include_top=False, input_tensor=model_input
    )
    x = resnet50.get_layer("conv4_block6_2_relu").output

    x = DilatedSpatialPyramidPooling (x)

    # ASPP, in fact, it is bilinear interpolation
    input_a = layers.UpSampling2D(
        size=(image_size[0] // 4 // x.shape[1], image_size[1] // 4 // x.shape[2]),
        interpolation= "bilinear",
    )(x)
```



F.Y.I. Details Inside ASPP

```
lgraph = deeplabv3plusLayers([224 224 3], 3, "resnet18")
analyzeNetwork(lgraph)
```

- **Atrous Spatial Pyramid Pooling** (ASPP) **pooling ≈ dilate rate**
  - In ASPP, **NO** pooling. Why call it ASPP? Pooling conceptually occurs using ***rate***
  - Require special Atrous "***pooling***" to = assemble **(aggregate)** different size feature maps

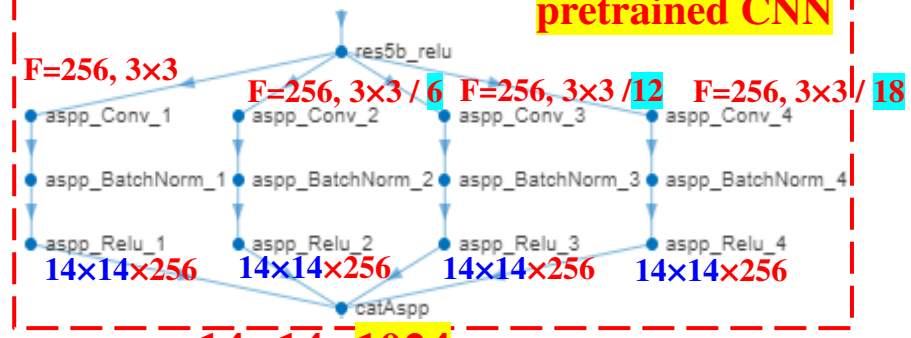


Atrous Spatial Pyramid Pooling (ASPP)

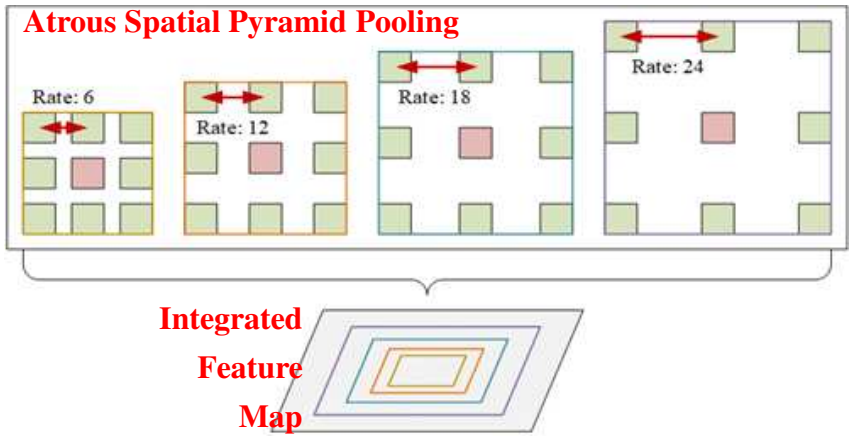
encoder

FM = 14 × 14 × 512 ● ● ●

Astrous inserted into pretrained CNN

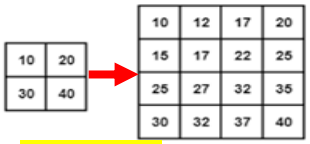


14×14×1024



decoder few ● ● ●

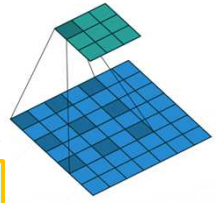
- **256**, **1×1** conv: 14×14×256
- **256**, 3×3 conv: 14×14×**256** **(learning)**
- Upsample by 16: **224×224**×256
- **3**, **1×1** conv: 224×224**×3 classes**
- Softmax: 224×224×3



bilinear interpolation

Dilated convolution animations

https://github.com/vdumoulin/conv_arithmetic



```
convolution2dLayer( filterSize, numFilters, DilationFactor=2)
```

3, 1, 2

Pre-Trained Semantic Segmentation, Google DeepLab V3+

- A DeepLab V3+ **already trained for segmentation** based on **Resnet18**
  - Download from **https://ssd.mathworks.com/supportfiles/vision/data/deeplabv3plusResnet18CamVid.zip**

```
deeplabv3plusLayers(inputImgSize, numClasses, "resnet18")
```

```
convolution2dLayer( filterSize, numFilters, DilationFactor=2)
```

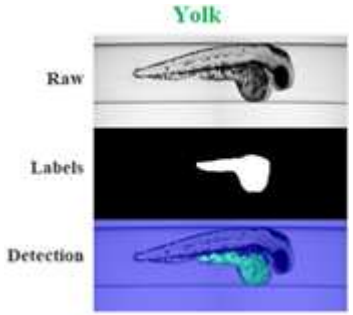
https://www.analyticsvidhya.com/blog/2019/02/tutorial-semantic-segmentation-google-deeplab/

- References for **Google DeepLab V3+**
  - **Keras** Multiclass semantic segmentation using DeepLabV3+
    - https://keras.io/examples/vision/deeplabv3_plus/

Folder: \FishEmbryo_Semseg_Programs

VAI_Slide_deeplabv3_plus_FishEmbryo.ipynb

VAI_Slide_ FishEmboyo_Deeplab_V3_Train.m



  - **Matlab** function https://www.mathworks.com/help/vision/ref/deeplabv3pluslayers.html
    - Semantic Segmentation Overview: https://www.mathworks.com/help/vision/examples/semantic-segmentation-using-deep-learning.html
    - **All kinds of pretrained object detectors: https://www.mathworks.com/help/vision/ug/choose-an-object-detector.htm**

Prepare Image ***Pairs***

Vision AI › Files › Course Examples + Data › VAI_Slide_FishEmbryo

FishEmbryo_All_Images.zip

- FishEmbryo_Label_Images_BMP
- FishEmbryo_Label_Images_PNG_for_Colab
- FishEmbryo_Raw_Images_Yolk

VAI_Slide_ImgPair_Prepare.ipynb

```
def load_and_process(img_path, mask_path):    # 1. read 1 img & 1 mask
    img = tf.io.read_file(img_path)   # Read and decode raw images
    img = tf.image.decode_png(img, channels=3)   # keras, do it explicitly
    img = tf.image.resize(img, IMG_SIZE)
    img = tf.cast(img, tf.float32) / 255.0  # Normalize to [0,1]

    mask = tf.io.read_file(mask_path)   # Read and decode masks
    mask = tf.image.decode_png(mask, channels=1) # Labels are grayscale
    mask = tf.image.resize(mask, IMG_SIZE, method="nearest")
    return img, mask
```

```
# 2. Create the Dataset, sort so pairs are in SAME order after glob
# glob = find wildcard names
image_paths = sorted(tf.io.gfile.glob(IMG_DIR + "*.png"))
mask_paths = sorted(tf.io.gfile.glob(MASK_DIR + "*.png"))

# create "store" & tell TF to slice (image, mask) pairs together
DS = tf.data.Dataset.from_tensor_slices((image_paths , mask_paths))
DS = DS.map(load_and_preprocess)  # to process pairs using this func.
DS = DS.map(AUG)     # to AUG pairs using this func.
```

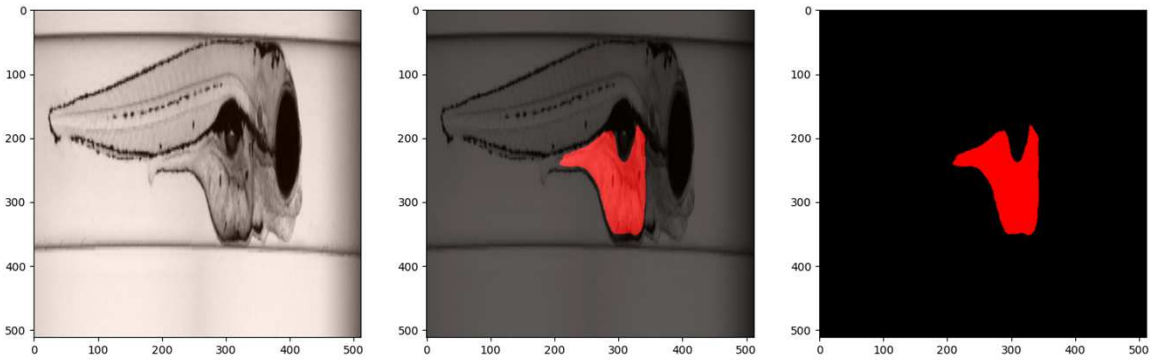
```
def  AUG(img, mask):
    if tf.random.uniform(()) > 0.5:
        img = tf.image.flip_left_right(img)
        mask = tf.image.flip_left_right(mask)
    img = bright(img,    max_delta=0.1)
    return img, mask
```

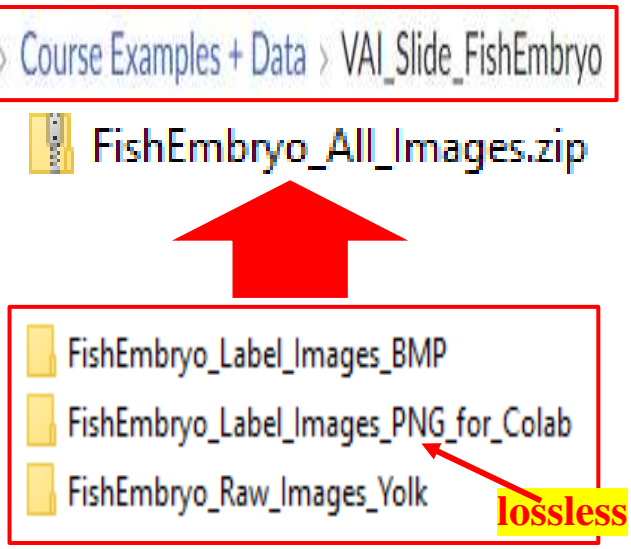
```
model.compile( ……..    4.
model.fit(
    train_DS,
    epochs=20,
    val_DS   # Prepared same way
)
```

```
# 3. Final Pipeline to create train / val / test datastore
train_DS = (
    DS.shuffle(100)         # sequential get from DS to fill buffer (100) for shuffle
    .batch(BATCH_SIZE)
    .prefetch(buffer_size=tf.data.AUTOTUNE)  # get next BAT when current on GPU
)
```

\VAI_Slide_FishEmbryo\VAI_Slide_deeplabv3_plus_FishEmbryo.ipynb

- https://keras.io/examples/vision/deeplabv3_plus/






```
train_images = sorted(glob(os.path.join(DATA_DIR, "FishEmbryo_Images/Yolk/*")))
train_masks = sorted(glob(os.path.join(DATA_DIR, "Label_Images_png/Yolk/*")))
```

```
def read_image(image_path, mask=False):
    image = tf.io.read_file(image_path)
    if mask:      # label images
        image = tf.image.decode_png(image, channels=1)
        image.set_shape([None, None, 1])
        image = tf.image.resize(images=image, size=IMAGE_SIZE)
    else:
        image = tf.image.decode_bmp(image, channels=3)
        image.set_shape([None, None, 3])
        image = tf.image.resize(images=image, size=IMAGE_SIZE)
        # Converting the images from RGB to BGR
        # Zero-centering each color channel with respect to the ImageNet dataset
        image = tf.keras.applications.resnet50.preprocess_input(image)
    return image
```

```
def load_data(image_list, mask_list):
    image = read_image(image_list)
    mask = read_image(mask_list, mask=True)
    return image, mask

def data_generator(image_list, mask_list):
    dataset = tf.data.Dataset.from_tensor_slices((image_list, mask_list))
    dataset = dataset.map(load_data, num_parallel_calls=tf.data.AUTOTUNE)
    dataset = dataset.batch(BATCH_SIZE, drop_remainder=True)
    return dataset

train_dataset = data_generator(train_images, train_masks)
```

Data Usage Agreement

- Images and data are provided by UFZ at Germany for non-commercial use
- Please do **NOT** share the data with others / public / internet
- Raw Images / Organ Images / Labeled Images
- Program templates (by Dr. Lai)

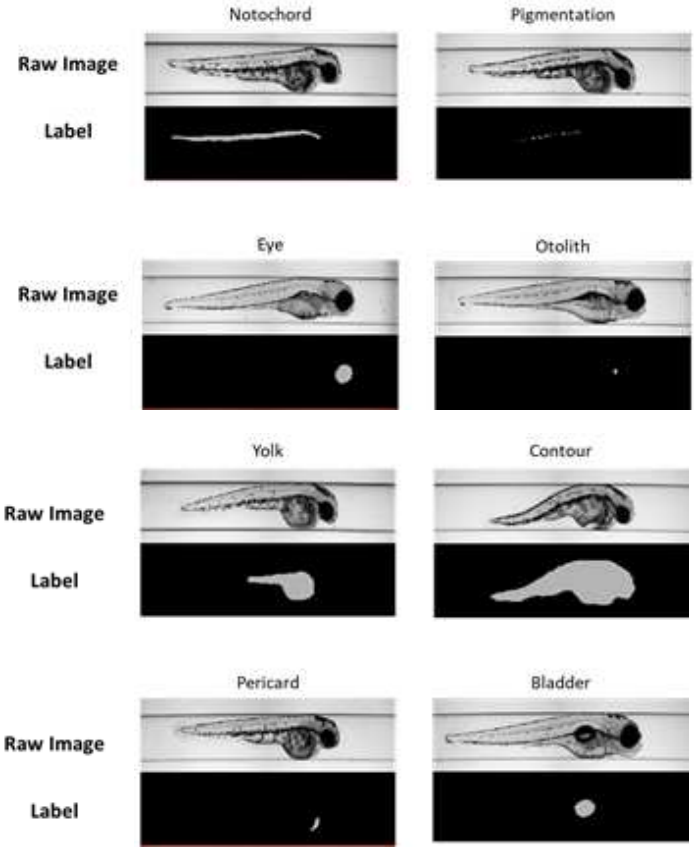
Vision AI › Files › Course Examples + Data › VAI_Slide_FishEmbryo

FishEmbryo_All_Images.zip

… and programs

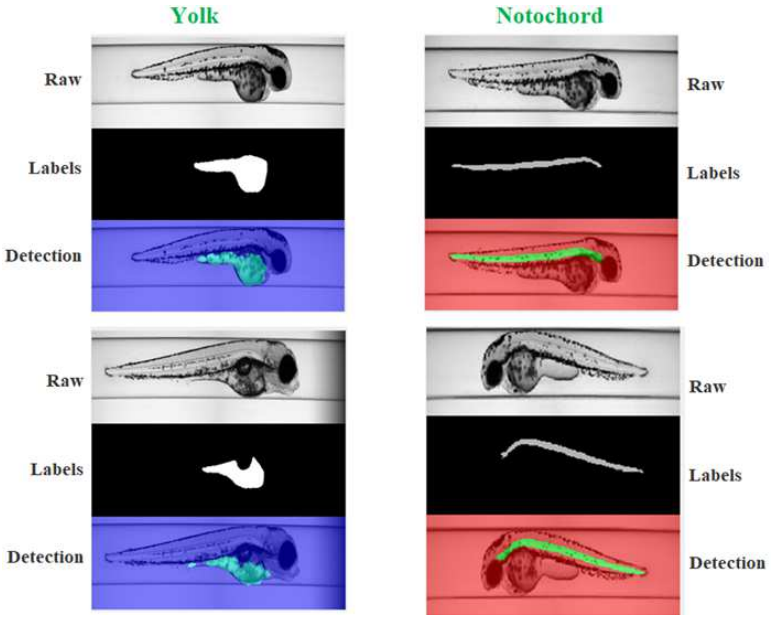
images+labels provided by Dr. Stefan Scholz

HELMHOLTZ CENTRE FOR ENVIRONMENTAL RESEARCH – UFZ



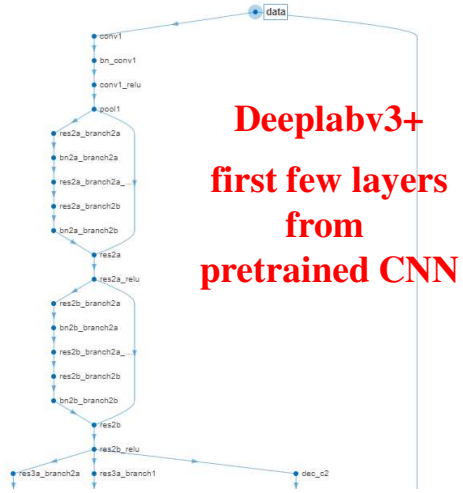
UNIVERSITY OF St.Thomas

HELMHOLTZ ZENTRUM FÜR UMWELTFORSCHUNG UFZ



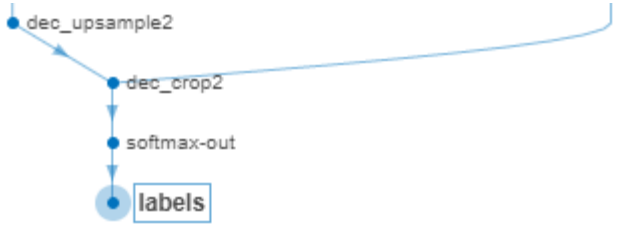
Why *Inverse Frequency*??

```
% FishEmboyo_Deeplab_V3_Train.m
lgraph = deeplabv3plusLayers(ImgSize, numClasses, "resnet18");

pxLayer = pixelClassificationLayer('Name', 'labels', 'Classes', ClassNames, ...
                                   'ClassWeights', inverseFrequency);
lgraph = replaceLayer(lgraph, "classification", pxLayer);
```



Deeplabv3+

first few layers from pretrained CNN



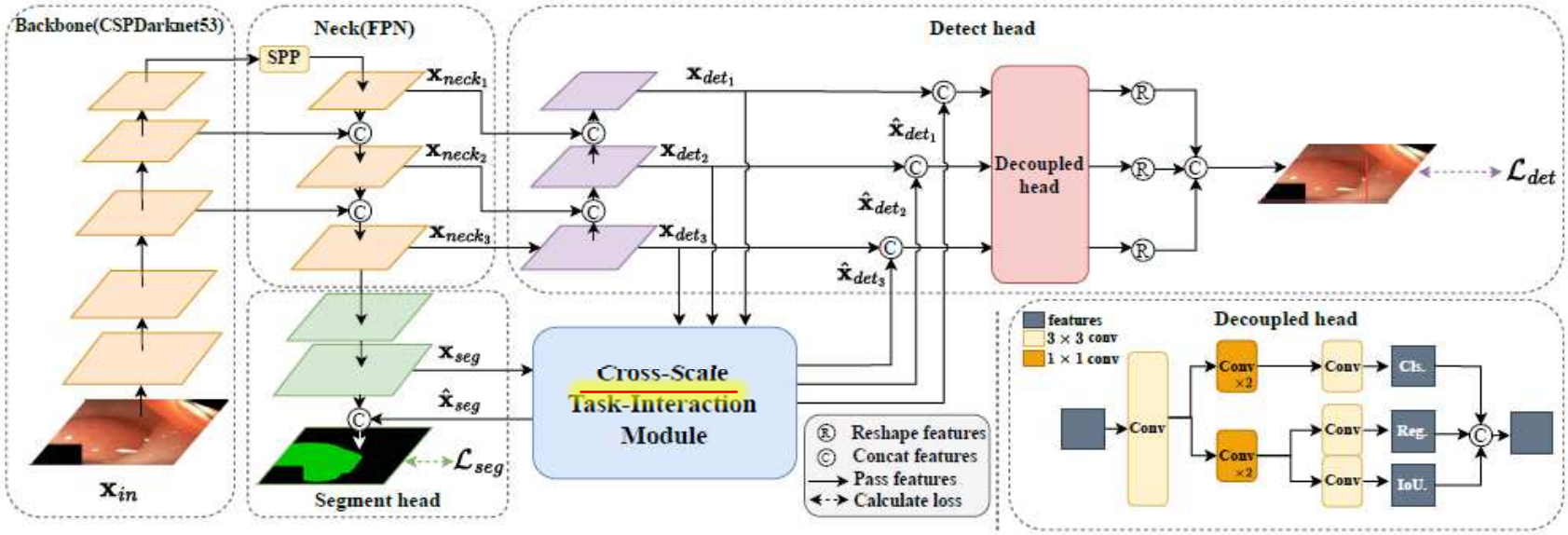
$$\frac{1}{\text{pixels freq of each label}} =$$

```
tbl = countEachLabel(pxds);
totalNumberOfPixels = sum(tbl.PixelCount);   ← pixel freq of each label
frequencyPxl = tbl.PixelCount / totalNumberOfPixels;
inverseFrequency = 1 ./ frequencyPxl;
```

```
for class_id, count in class_counts.items():
    weights[class_id] = total_pixels / (num_classes * count)

model.fit(x, y, class_weight=weights, ...)   # inverseFrequency
```

2024 Paper vs. Google DeepLab V3+

YOLO-MED : MULTI-TASK INTERACTION NETWORK FOR BIOMEDICAL IMAGES



cross-scale analysis with **"Pooling"** AND **NO** discussing of noise removal

- Instead, use **Google DeepLab V3+** / Semantic Segmentation, **WHY???**
  - **Special *atrous* convolution➔** better ***FOV (field of view)*** i.e. better ***cross-scale*** **analysis**
  - **Less aggressive** pooling (**NOT** from start to end) with **special convolution**
    - "**pooling**" causes ↓resolution **&** info loss
    - "**pooling**" is OK for classifying image, **bad** for pixel segmentation

Outline

- Object Detection, Fast RCNN
- Object Detection, Semantic Segmentation, Labeling, Applications
- Encode + Decode CNN, Transposed Convolution, Pre-Trained CNN
  - **SegNet, U-Net** Family, **Google's DeepLab V3+,** **Meta's SAM** √
- Quality Evaluation of Object Detection
- Identifying Morphological Changes in Organs w/ Occlusion Map

Comparison, All to Preserve Image Detail for Precise Object Locations

- **They are for different purposes, not just a linear upgrade over years**

Feature	DeepLabV3+ (2018)	U-Net3+ (2015 / 2019 / 2020)	SAM / Segment Anything (2023)
Architecture Type	CNN-based, NO decoder, with **dilated convolutions** / **Atrous** Convolutions	CNN **encoder–decoder** with **full-scale skip connections** (dense connections across all scales)	Transformer-based **ViT** with multiple **encoder** + lightweight mask **decoder**
Generalization	Moderate → needs retraining/fine-tuning for new domains	Moderate → needs retraining/fine-tuning for new domains	Very high → **zero-shot** or prompt-based segmentation **(but = regions)**
Prompting	No user prompt required (other than labels)	No user prompt required (other than labels)	Supports user prompts (points, boxes) for **interactive *region* segmentation**
Parameter Size	~40–60M (depending on backbone)	~25–40M	Very large: **~600M** (ViT-H encoder) Trained on 11M images, 1 billion labels, high resolution images
Strengths	Strong semantic segmentation; robust to scale variation (via ASPP); good for natural images **Faster** for real-time surgery w/ upgrade	Excellent multi-scale fusion; strong performance on biomedical & fine-structure segmentation	Foundation model; **highly generalizable** for **regions**; interactive segmentation; strong zero-shot ability
Weaknesses	Rough boundary precision; No instance separation	**Heavy computing**; slower due to **full-scale skip,** No instance separation	**Very large model**, **memory intensive**; **domain gap** on medical images without adaptation, No instance separation